Py-CoSim: an open-source, pure-Python framework for synchronized co-simulation of EnergyPlus buildings with energy, mobility and artificial-intelligence components

Abstract

Buildings are increasingly operated as part of larger systems that include distributed energy resources, electricity tariffs and grids, occupants' travel behavior and data-driven or artificial-intelligence (AI) controllers. Studying them requires co-simulation, yet established platforms demand expertise in several languages and middleware, while Python-based tools typically address a single agent-environment loop. This paper presents Py-CoSim, an open-source framework in which any Python model and any number of EnergyPlus buildings are coupled on one synchronized clock. Py-CoSim (i) inverts EnergyPlus' callback-driven execution into a lock-step, steppable component that runs in an isolated worker process, (ii) schedules heterogeneous components with a graph-based master algorithm that orders direct-feedthrough dependencies, detects algebraic loops and supports multi-rate coupling, and (iii) records every exchange on a time-indexed data bus. Verification against stand-alone EnergyPlus shows bit-identical results (maximum absolute difference 0.0) and exact equivalence between Python actuation and native schedules; parallel worker processes reproduce serial results exactly and accelerated multi-building runs by up to 2.9 times on a four-core laptop. Three case studies demonstrate the framework: a ten-building community with photovoltaics and a shared battery, where coordinated pre-cooling and storage reduced on-peak grid import by 83% and energy cost by 33%; a large-language-model supervisory controller whose guardrails kept all set-points within bounds under 50% injected faults; and travel-diary-driven occupancy for 23 apartments, where occupancy-responsive thermostats

saved 5.7% and 6.0% of cooling electricity for long- and short-commute households, depending on household composition as much as on commuting.

Keywords

co-simulation; EnergyPlus; Python; grid-interactive efficient buildings; large language models; occupant behavior

1 Introduction

Buildings are no longer isolated energy consumers. Grid-interactive efficient buildings are expected to shift and shed load in response to prices and grid conditions, to host photovoltaics (PV), batteries and electric-vehicle (EV) chargers, and to coordinate with neighbouring buildings (Neukomm et al. 2019; Hong et al. 2025). Their operation increasingly depends on models from other disciplines: weather and solar resources, tariffs and distribution grids, occupants' presence and mobility, and data-driven, model-predictive or AI-based controllers (Yan et al. 2015; Drgoňa et al. 2020; Wang and Hong 2020; Zhang et al. 2021). Answering questions at these interfaces – for example how pre-cooling and a community battery should be coordinated under a time-of-use tariff, or how commuting patterns change the value of a smart thermostat – requires running detailed building energy models together with such models, exchanging data at every time step. This is co-simulation (Gomes et al. 2018; Trčka et al. 2009), and its importance for building simulation has long been recognized (Wetter 2011; Hong et al. 2018).

A rich ecosystem of co-simulation tools exists. The Building Controls Virtual Test Bed (BCVTB) couples EnergyPlus with Modelica, MATLAB and other programs through Ptolemy II (Wetter 2011); the Functional Mock-up Interface (FMI) standardizes model exchange and co-simulation and is supported by EnergyPlus and the Modelica Buildings library (Blochwitz et al. 2011; Nouidui et al. 2014; Wetter et al. 2014); Spawn couples EnergyPlus envelopes with Modelica HVAC and controls (Wetter et al. 2020); mosaik and

HELICS orchestrate federated smart-grid co-simulations (Steinbrink et al. 2019; Palmintier et al. 2017; Hardy et al. 2024); and BOPTEST and Alfalfa expose building models as services for control benchmarking (Blum et al. 2021; Benne et al. 2024). These platforms are powerful, but they require users to master several tools, languages, model-export toolchains or middleware, which is a considerable barrier for building scientists who mainly work in Python. At the other end of the spectrum, Python environments for reinforcement learning (RL) – Gym-EnergyPlus, Sinergym, COBS, Energym, BuildingGym and CityLearn (Moriyama et al. 2018; Zhang and Lam 2018; Jiménez-Raboso et al. 2021; Zhang and Ardakanian 2020; Scharnhorst et al. 2021; Dai et al. 2025; Vázquez-Canteli et al. 2019; Nweye et al. 2025) – are easy to use but are organized around a single agent interacting with a single environment rather than around a general network of models.

Since version 9.3, EnergyPlus (Crawley et al. 2001) exposes a Python application programming interface (API) through which external Python code can read sensors and meters and write actuators while EnergyPlus runs (DOE 2022). This API has made Python-EnergyPlus integration markedly easier (Bonnema et al. 2025; Chinde et al. 2025; Campoy-Nieves et al. 2025), and an earlier prototype of the present work, Py-CoSim 0.x (Zhang et al. 2023), used it to connect EnergyPlus with deep-learning occupancy models and a large language model (LLM). That prototype established feasibility but also exposed the obstacles that any general Python co-simulation framework built on the EnergyPlus API must overcome:

- **Inverted control flow.** EnergyPlus owns its time loop and calls user code back; orchestrating several EnergyPlus models and other components on a common clock requires turning EnergyPlus into a component that an external master can advance one step at a time.

- **Concurrency and isolation.** As documented in Section 3.3, the EnergyPlus API (tested with version 22.2) deadlocks when several simulations run concurrently in threads of one process and cannot start a second simulation in a process after a first one has finished, so naïve multi-building designs fail.
- **Execution semantics.** Without an explicit definition of which values a component sees at a given time, of the execution order and of how feedback loops are broken, co-simulation results are hard to interpret and reproduce (Kübler and Schiehlen 2000; Gomes et al. 2018).
- **Data exchange and auditability.** Py-CoSim 0.x exchanged data through one CSV file per component and time step and synchronized buildings by counting files, which is slow, fragile and leaves no structured record – a particular concern when LLMs or other AI agents act on a building (Liu et al. 2025).

This paper presents Py-CoSim, a complete redesign that addresses these obstacles. Its contributions are:

(1) A component abstraction with explicit feedthrough semantics and multi-rate step sizes, under which EnergyPlus models, physical models, data sources, controllers, machine-learning (ML) models, LLM agents and RL agents are interchangeable, and any plain Python function is a valid component.

(2) A lock-step protocol that makes EnergyPlus steppable inside its own callbacks, with per-step time-stamp verification, isolated worker processes that enable parallel multi-building co-simulation, and automatic synchronization of run period, time step, sizing and daylight-saving settings.

(3) A graph-based master algorithm (topological scheduling, algebraic-loop detection, delayed and aggregated connections) together with an in-memory data bus that records every exchange, events and errors.

(4) A verification showing bit-identical agreement with stand-alone EnergyPlus and a characterization of overhead and parallel scalability.

(5) Three case studies – a grid-interactive community, LLM-in-the-loop control with guardrails, and travel-behavior-driven occupancy – released, together with six further examples, as runnable templates.

Section 2 positions Py-CoSim among existing tools. Section 3 describes the framework, Section 4 its verification and performance, and Section 5 the case studies. Section 6 discusses lessons, limitations and future work, and Section 7 concludes.

2 Related work and positioning

Co-simulation couples simulators that each solve their own sub-problem and exchange data at communication points (Gomes et al. 2018). Two coupling schemes are classical: in a Jacobi (parallel) scheme all subsystems use inputs from the previous communication point, whereas in a Gauss-Seidel (sequential) scheme subsystems are evaluated in an order such that later ones use the current outputs of earlier ones (Kübler and Schiehlen 2000). Building-specific studies have shown how the choice of coupling and of the communication step affects accuracy and stability (Trčka et al. 2009). A framework therefore needs to make these choices explicit.

Table 1 summarizes representative tools. General-purpose co-simulation frameworks (BCVTB, FMI-based toolchains, mosaik, HELICS) are the most expressive, but EnergyPlus participates through external interfaces, exported FMUs or federates, and users must configure middleware or additional languages (Wetter 2011; Nouidui et al. 2014; Steinbrink et al. 2019; Hardy et al. 2024). Benchmarking services (BOPTEST, Alfalfa) expose curated building emulators through web interfaces and are excellent for comparing controllers, but they are not designed to embed arbitrary user models such as a battery fleet or a travel-behavior model inside the simulation loop (Blum et al. 2021; Benne et al. 2024). RL

environments wrap EnergyPlus (increasingly through its Python API) or data-driven building models in the Gym/Gymnasium interface (Towers et al. 2025); they couple one learning agent with one environment and typically leave other components to be hard-coded in the environment (Campoy-Nieves et al. 2025; Dai et al. 2025; Nweye et al. 2025). Urban- and district-scale platforms such as URBANopt focus on modelling workflows for many buildings rather than on step-wise coupling with external models (El Kontar et al. 2020; Reinhart and Cerezo Davila 2016; Hong et al. 2020; Allegrini et al. 2015). Digital-twin platforms couple EnergyPlus with live data and FMUs for specific deployments (Li and Hong 2025). Py-CoSim targets the gap between these groups: a pure-Python co-simulation framework in which EnergyPlus is a first-class, steppable component and in which any Python model – physical, data-driven or AI-based – can be connected with explicit, verifiable semantics.

Table 1 Representative co-simulation and control-testing tools for buildings (characteristics as reported in the cited publications)

Tool	Main purpose	User-facing language / set-up	EnergyPlus coupling	Other components
BCVTB (Wetter 2011)	General co-simulation middleware	Ptolemy II models, tool adapters	External interface over sockets	Modelica, MATLAB/Simulink, Radiance, …
FMI / Modelica Buildings, Spawn (Nouidui et al. 2014; Wetter et al. 2014, 2020)	Model exchange and co-simulation standard; Modelica HVAC/controls	Modelica or FMU export toolchains	FMU import/export; Spawn coupling	Any FMU
mosaik, HELICS (Steinbrink et al. 2019; Hardy et al. 2024)	Smart-grid / multi-domain co-simulation	Python (mosaik); multi-language federates (HELICS)	Through user-written adapters	Grid, communication, market simulators
MLE+ (Bernal et al. 2012)	Control design and deployment	MATLAB/Simulink	EnergyPlus external interface	MATLAB toolboxes
BOPTEST, Alfalfa (Blum et al. 2021; Benne et al. 2024)	Control benchmarking, virtual buildings	Web (REST) clients	BOPTEST: Modelica emulators; Alfalfa: OpenStudio/EnergyPlus or Modelica	External controllers
Sinergym, BuildingGym, COBS (Campoy-Nieves et al. 2025; Dai et al. 2025; Zhang and Ardakanian 2020)	RL environments	Python (Gym/Gymnasium)	EnergyPlus-based (Sinergym, BuildingGym via the Python API)	One learning agent per environment
CityLearn (Nweye et al. 2025)	Community-scale RL	Python (Gymnasium)	Data-driven building models	Batteries, PV, EVs, heat pumps
Py-CoSim 0.x (Zhang et al. 2023)	Prototype Python EMS co-simulation	Jupyter notebooks	Python API, threads	CSV-file exchange
Py-CoSim (this work)	General Python co-simulation	Python, YAML, Gymnasium	Steppable component in worker processes	Any Python model; library of 17 ready-

				made components

3 The Py-CoSim framework

Figure 1 shows the architecture. A scenario – written as a Python script, a Jupyter notebook, a YAML file or created by an RL training loop – instantiates components and connects their ports. The CoSimulation master owns the clock, schedules components, routes data through the data bus and supervises the run. EnergyPlus components run in worker processes; all other components run in the master process.

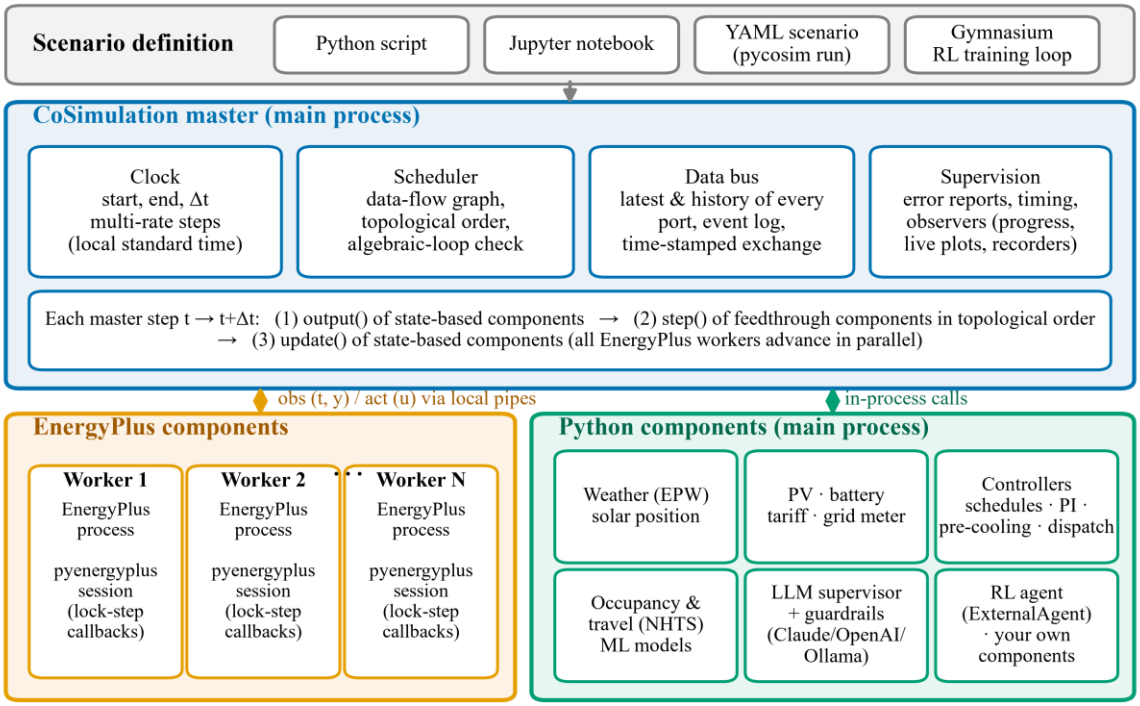


Fig. 1 Architecture of Py-CoSim. Scenarios instantiate and connect components; the master executes a three-phase step on a common clock; each EnergyPlus model runs in its own worker process, other components in the main process.

3.1 Design principles

Five principles guided the design. (1) *Python only*: users write components as Python functions or classes; no other language, middleware or model-export step is needed. (2)

Explicit semantics: the value each component sees at each time, the execution order and the treatment of feedback loops follow formal rules that are documented, verified and reported back to the user. (3) *Isolation and parallelism*: every EnergyPlus instance runs in its own process, which both circumvents the API's concurrency limitations and uses all processor cores. (4) *Reproducibility and auditability*: random numbers are seeded per component, every exchanged value and every event is recorded, and AI decisions can be recorded and replayed. (5) *Progressive disclosure*: a first co-simulation takes a dozen lines, while advanced users can control every aspect (custom components, step sizes, delayed links, observers, executors).

3.2 Component abstraction

A component i is a discrete-time system with input ports u, output ports y and an internal state x, executed every m_i master steps of length Δt. Py-CoSim distinguishes two kinds. A *feedthrough* component may use its inputs at time t_k to compute its outputs at the same time:

$$y_i(t_k) = h_i(x_i(t_k), u_i(t_k)), \;\; x_i(t_{k+m_i}) = f_i(x_i(t_k), u_i(t_k)) \tag{1}$$

Controllers, tariffs, data readers, PV and battery models, ML predictors and LLM agents are of this kind. A *non-feedthrough* (state-based) component reports outputs that depend only on its state:

$$y_i(t_k) = g_i(x_i(t_k)), \;\; x_i(t_{k+m_i}) = f_i(x_i(t_k), u_i(t_k)) \tag{2}$$

Physical simulators – EnergyPlus and the resistance-capacitance (RC) building model of the library – belong to this kind: the zone temperature reported at t_k results from the past, while the set-point received at t_k acts on the interval $[t_k, t_k + \Delta t)$. In code, feedthrough components implement step(t, inputs); non-feedthrough components implement output(t) and update(t, inputs). Between two executions a component's outputs are held (zero-order hold):

$$y_i(t_k) = y_i(t_{m_i \lfloor k/m_i \rfloor}), \;\; m_i \in \mathbb{N} \tag{3}$$

so that, for example, an LLM supervisor can decide hourly while EnergyPlus advances every 15 min (Fig. 2b). The simplest component is a decorated function whose parameters become

input ports and whose default values become input defaults; classes add state, set-up and clean-up logic. Each component receives a context object giving access to the clock, to the history of any port on the data bus, to structured logging, to a private output folder and to a random-number generator seeded from the simulation seed and the component name, which makes stochastic components reproducible and independent of the order in which they were added.

3.3 Making EnergyPlus a steppable component

In library mode, EnergyPlus calls registered Python functions at fixed points of its time loop (DOE 2022). Py-CoSim registers two callbacks and turns them into a lock-step handshake with the master (Fig. 2a). At the end of each zone time step (after zone reporting) the worker reads all requested output variables, meters and internal variables into a snapshot. At the beginning of the next zone time step (after the heat-balance initialization) it sends the snapshot with a time stamp to the master and blocks until it receives the actuator values for the step; it then applies them – an actuator value of None releases the actuator back to EnergyPlus' own schedules and controls – and returns control to EnergyPlus, which simulates the interval. Design-day, sizing and warm-up periods are passed through without blocking.

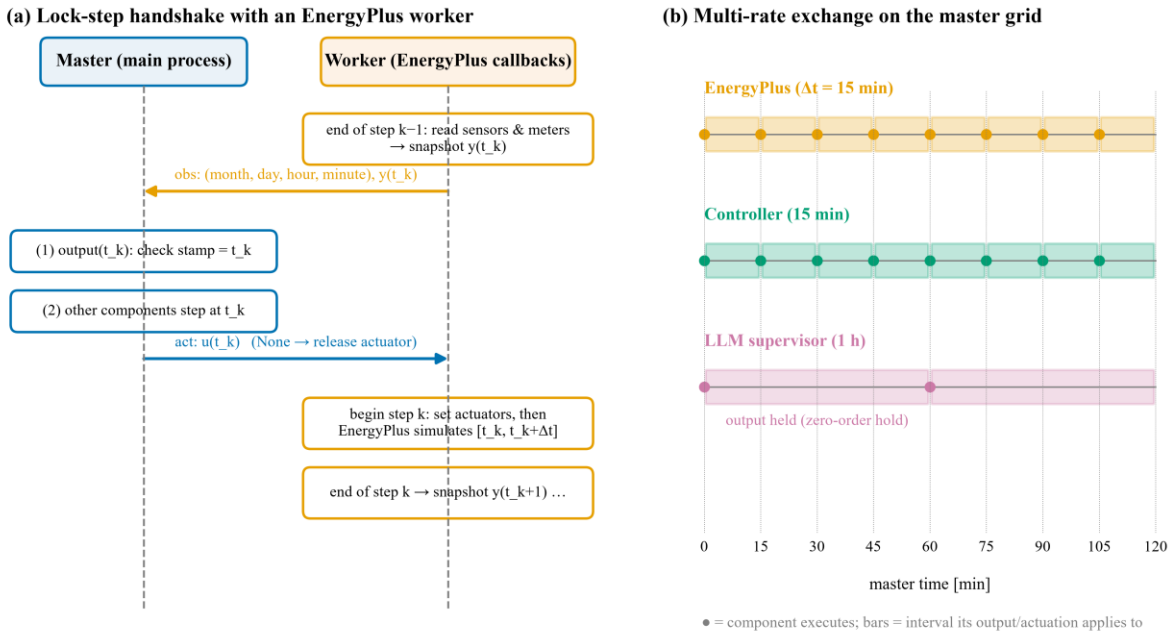


Fig. 2 (a) Lock-step handshake between the master and an EnergyPlus worker; (b) multi-rate execution on the master grid with zero-order hold of slower components.

Two details matter for correctness. First, the value published at master time t_k must be the value EnergyPlus itself reports with time stamp t_k, i.e., the state at the end of the previous interval; reading outputs at the end-of-step callback, rather than at the beginning of the next step, achieves this for zone variables, meters and weather variables alike (variables that EnergyPlus reports at the HVAC system time step are sampled at the last system sub-step). Second, the time stamp must be derived robustly. The API's current-time and minute functions follow EnergyPlus' adaptive HVAC sub-steps and therefore return values such as 0:38 h within a 15-min zone step; Py-CoSim instead builds the time stamp of interval k from the date, the hour h and the zone time-step number n_z,

$$t_k = d + h + (n_z - 1)\,\Delta t, \tag{4}$$

and the master verifies at every step that it equals its own clock, raising a synchronization error otherwise. Before a run, the component rewrites the model – converted to epJSON with EnergyPlus' own converter, so that fields are addressed by name – to match the co-simulation: time steps per hour from Δt, a run period with an explicit calendar year from the simulation

window, optionally sizing periods derived from the weather file's extreme periods, and optionally the removal of daylight-saving time. The latter proved important: the co-simulation clock, EnergyPlus' internal clock and EPW weather data use local standard time, whereas EnergyPlus shifts schedules by one hour when daylight-saving time is defined.

Handles are resolved once, at the first weather-file time step. If a sensor, meter or actuator name cannot be resolved, the worker retrieves the model's catalogue of available API data and the master reports the closest matches ("did you mean …"); the same catalogue is available as a searchable table through a command-line tool. This addresses one of the most frequent stumbling blocks in practice.

Finally, EnergyPlus runs in a separate Python interpreter per component. We tested the alternatives with EnergyPlus 22.2: four simulations started concurrently in threads of one process deadlocked before reading their inputs, and a second simulation started in a process after a first one had completed stalled at start-up, although each used a new API state. Worker processes avoid both problems, run truly in parallel, and contain crashes. Workers are launched as separate interpreters rather than through Python's multiprocessing module so that user scripts need no main-module guard and run unchanged in Jupyter. Master and worker communicate through an authenticated local pipe. Start-up is split into a non-blocking launch and a blocking wait, so the sizing and warm-up periods of all buildings proceed in parallel. An in-process executor remains available for single-building debugging; because EnergyPlus can run only once per process, later in-process runs automatically fall back to a worker process.

3.4 Master algorithm

Connections link output ports to input ports. A connection may carry a transformation (e.g., W to kW), may aggregate several sources with a reducer (e.g., the sum of building loads) and

may be marked as delayed, in which case the target reads the value the source had at the end of the previous master step:

$$u_j(t_k) = \varphi_c(\rho_c(y_{s_1}(t_{k-d_c}), \ldots, y_{s_n}(t_{k-d_c}))), \quad d_c \in \{0, 1\} \tag{5}$$

where ρ_c is the reducer, φ_c the transformation and d_c the delay of connection c. Only non-delayed connections between two feedthrough components impose an ordering constraint, because outputs of non-feedthrough components are known at the start of each step. The master builds the corresponding directed graph G = (F, E) over the set F of feedthrough components and sorts it topologically with Kahn's algorithm (Kahn 1962), breaking ties by a user-settable priority and then by insertion order so that the schedule is deterministic. A cycle in G is an algebraic loop; the master reports the cycle and suggests either a delayed connection (a Jacobi-type coupling of that link) or a non-feedthrough formulation. Each master step then executes three phases:

(1) *output* – every due non-feedthrough component reports y(t_k) (for EnergyPlus, the master receives the snapshots of all workers);

(2) *step* – due feedthrough components run in topological order, each receiving its inputs per Eq. (5);

(3) *update* – every due non-feedthrough component receives its inputs and advances to t_k + Δt (for EnergyPlus, actuator values are sent to all workers, which then simulate the interval concurrently).

Within a step, the scheme is therefore Gauss-Seidel along the feedthrough graph and explicit (non-iterative) across non-feedthrough components, with optional Jacobi links where the user chooses them. Steps can be executed one by one, which is used for interactive exploration in notebooks and by the RL interface, which pauses the step right before an external agent acts.

3.5 Data bus, results and supervision

The data bus replaces the per-step CSV files of Py-CoSim 0.x. It keeps the latest value of every output port, an optional snapshot of the previous step for delayed connections, and the full history of each port indexed by master step. Components can query any port's latest value or history, e.g., to build lagged features for an ML model. At the end of a run, the history is returned as a tidy table on the master grid (with zero-order hold for slower components), together with a structured event log (warnings, component messages, every LLM prompt and response), per-component timing statistics and metadata (versions, connections, EnergyPlus warnings and errors). If any component fails, all workers are terminated, partial results are saved and an error report records the failing component, phase, simulation time, the inputs it received, the recent events and the connection list. Observers can watch a run without taking part in it; built-in observers draw live (online) plots, stream values to a CSV file and show progress.

3.6 Component library and interfaces

Table 2 lists the component library, which provides the building blocks most often needed in grid-interactive, occupant-centric and AI-related building studies. The PV model follows the structure of PVWatts (Dobos 2014) with an isotropic-sky transposition, and the weather component interpolates EPW data exactly as EnergyPlus does, so that weather values seen by Python components coincide with those inside EnergyPlus. Beyond components, Py-CoSim offers: a Gymnasium environment (Towers et al. 2025) that exposes any co-simulation to RL libraries through an ExternalAgent component whose inputs are the observations and whose outputs are the actions (a trained policy can be plugged into the same scenario for evaluation); a YAML scenario format and command-line runner; and model-editing utilities (run period, time step, outputs, sizing, floor-area scaling).

Table 2 Built-in components (all are ordinary components and can be replaced or extended)

Category	Components	Main inputs → outputs
Buildings	EnergyPlusComponent; RCBuilding (2R2C, ideal HVAC)	actuators → sensors, meters; set-points, weather, gains → temperatures, powers
Weather and data	EPWWeather (+ solar position); TimeSeriesSource; DailyProfile; Constant	– → temperature, irradiance, wind; any time series
Energy resources and grid	PVSystem; Battery; TimeOfUseTariff; GridMeter	irradiance → AC power; power set-point → power, SOC; – → price; net power → energy, cost, peak
Controls	ThermostatSchedule; PrecoolingController; PIController; PeakShavingController	time, measurements → set-points, battery dispatch
People and mobility	HouseholdOccupancy (NHTS diaries); OccupancyMLP	– → people at home per dwelling
AI decision making	LLMSupervisor with Anthropic, OpenAI-compatible, Ollama, replay and fault-injection back-ends; ExternalAgent (Gymnasium)	measurements, intent → bounded set-points

Two components deserve detail. The LLM supervisor treats model output as untrusted. Each decision builds a prompt containing the operator's intent and the current measurements, and passes the response through a guardrail pipeline: extraction of the first JSON object; validation that only declared set-points with finite numeric values are used; clipping to [u_min, u_max]; rate limiting,

$$u_k = u_{k-1} + \text{sign}(\hat{u}_k - u_{k-1}) \min(|\hat{u}_k - u_{k-1}|, \Delta u_{\max}), \quad (6)$$

where û_k is the clipped LLM proposal; and a fallback that holds the last valid set-point on timeouts, refusals or unusable answers. Prompts, raw responses, applied values and guardrail interventions are written to the event log, and a recording back-end stores exchanges so that a run can be replayed exactly without calling the model again. The household-occupancy component draws households from 13,548 travel diaries of the 2017 U.S. National Household Travel Survey (FHWA 2017) (5,548 households owning a hybrid, electric or alternative-fuel vehicle and a random sample of 8,000 other households). A person is assumed at home before the first departure and after the last return of the diary day; household members without trips stayed home; and a household's commute proxy is the duration of the first trip of its earliest-leaving member. This corrects the prototype (Zhang et al. 2023), which counted people as at

home whenever they were not travelling. Each dwelling keeps its household size and draws a new diary day of a same-sized household every midnight, which reproduces day-to-day variability in the spirit of stochastic occupancy models (Page et al. 2008; Chen et al. 2018).

4 Verification and performance

All experiments used the U.S. Department of Energy small-office reference building (511 m², five conditioned zones, packaged single-zone air conditioners with gas furnaces) (Deru et al. 2011), the Tucson, Arizona TMY3 weather file, EnergyPlus 22.2, a 15-min step, and the week of 17–23 July 2023 (672 steps) unless stated otherwise. Timings were measured on a laptop (11th Gen Intel Core i7-1185G7 @ 3.00GHz, 4 physical cores / 8 logical processors, 16 GB memory) under Windows 11 with Python 3.12.

4.1 Fidelity of the EnergyPlus wrapper

Three tests were performed (Fig. 3). V1 (non-intrusiveness): the model was run once from the EnergyPlus command line and once inside Py-CoSim with sensors and meters only; the time-step outputs of both runs written by EnergyPlus to its SQL file (7 series: five zone temperatures, outdoor temperature, facility electricity) differed by at most 0.0, i.e., they are bit-identical. V2 (exchange timing): every value Py-CoSim published at master time t_k was compared with the value EnergyPlus reported with time stamp t_k; the maximum absolute difference was 0.0. V3 (actuation equivalence): a pre-cooling schedule (22 °C from 13:00 to 16:00 and 26.5 °C from 16:00 to 21:00 on weekdays) was implemented once as a native EnergyPlus schedule and once as a Python function writing the cooling set-point through an actuator; the two runs differed by at most 0.0 in every output, while the pre-cooling itself changed the core-zone temperature by up to 2.5 K relative to the original schedule. V3 also exposed the daylight-saving shift discussed in Section 3.3: with the model's original daylight-saving setting, the native schedule acted one hour earlier than the Python controller, which keeps standard time.

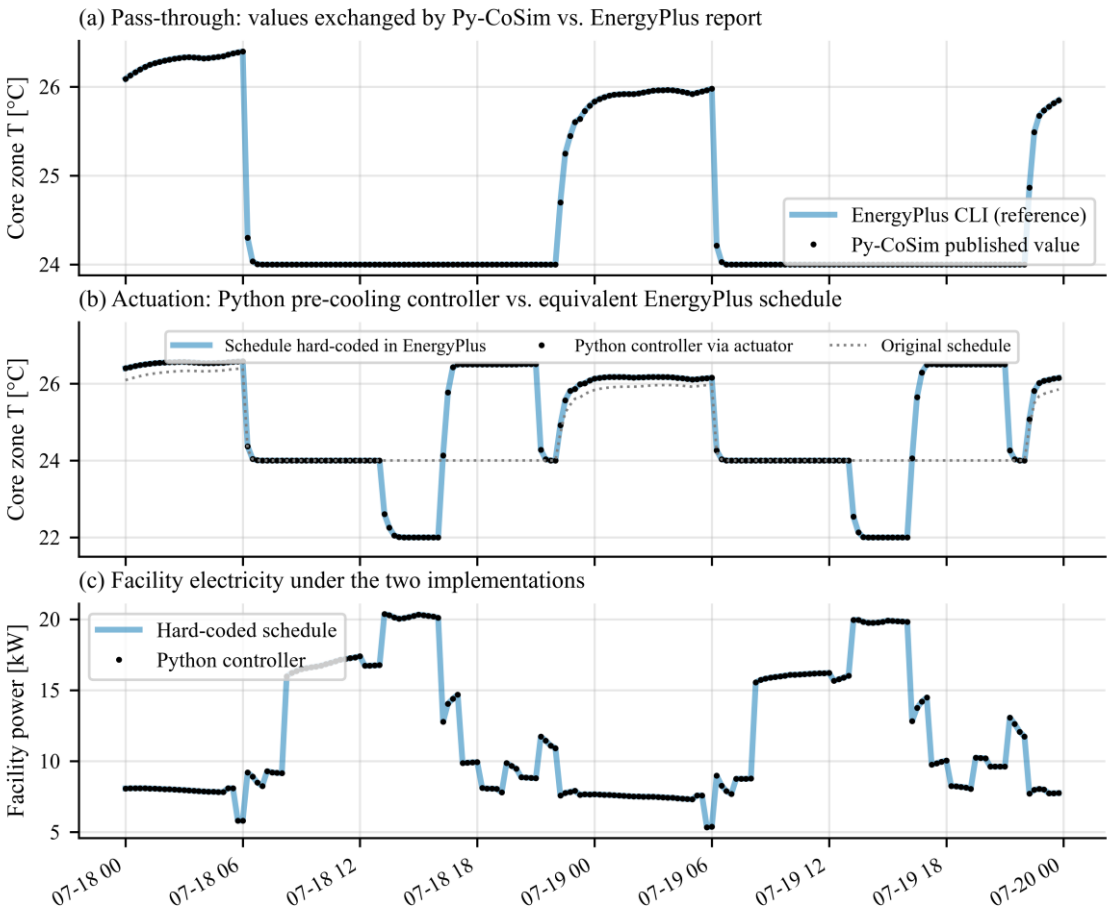


Fig. 3 Verification of the EnergyPlus component (two of seven days shown): (a) values exchanged by Py-CoSim versus EnergyPlus' own report; (b, c) a Python pre-cooling controller acting through an actuator versus the equivalent schedule hard-coded in EnergyPlus.

4.2 Overhead and scalability

For one building and one week, the EnergyPlus command line took 4.5 s on average (three repetitions), Py-CoSim with a worker process 8.3 s and with the in-process executor 7.5 s, including model preparation, sizing and warm-up; run-to-run variation on the laptop was about ±1 s. Repeating the comparison for 1, 7 and 28 days (nine runs per mode) and fitting wall time against the number of steps separated the overhead into a fixed part of about 0.9 s per run (model preparation, launching the worker interpreter, EnergyPlus running in library mode) and 1.5 ms per exchanged step, which is small compared with the cost of a typical

controller or ML model. The master itself adds 23 µs per component and step for trivial Python components. Exchanging one value through the data bus takes 3.33 µs, against 30.6 ms for writing and reading one CSV file as in Py-CoSim 0.x, a factor of about 9,000.

For scalability, N = 1 to 16 variants of the office (floor areas scaled by ±10%) were co-simulated with one Python controller actuating every building at every step, either serially (one co-simulation per building) or in a single co-simulation with parallel workers (Fig. 4). Parallel execution was 2.7 times faster for eight buildings and 2.9 times faster for 16 buildings (39 s versus 115 s). Since EnergyPlus is CPU-bound, the speed-up saturates at about 74% of the ideal value for the 4 physical cores of the test laptop (hyper-threads, the master process and memory bandwidth limit the rest); workstations with more cores scale further. In every case the parallel results were identical to the serial ones (maximum absolute difference 0.0), confirming that workers are fully isolated.

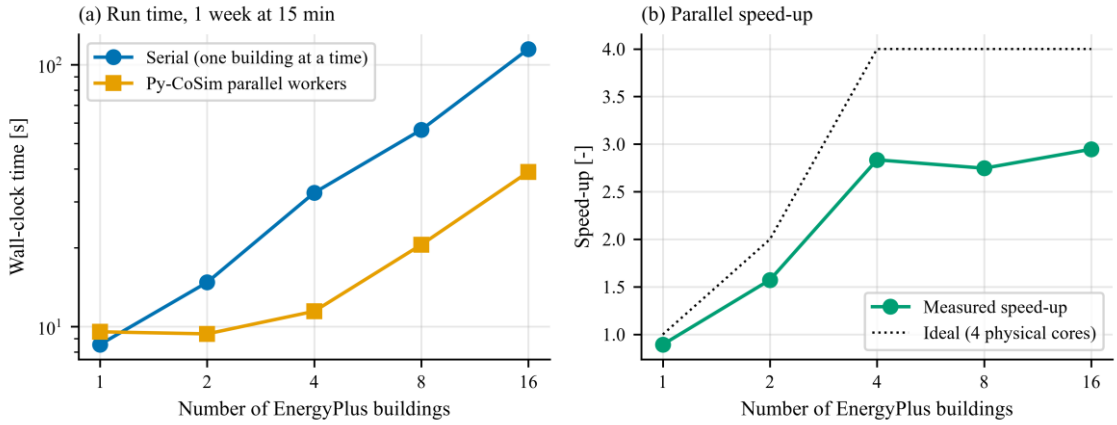


Fig. 4 Wall-clock time (a) and speed-up (b) of co-simulating N EnergyPlus buildings for one week at 15-min steps, serially and with Py-CoSim's parallel worker processes.

5 Case studies

The three case studies are distributed as runnable examples and can be reproduced with one command each. They are designed to exercise different capabilities rather than to provide definitive answers to the domain questions they touch.

5.1 Case A: a grid-interactive community

Ten small offices (each a separate EnergyPlus instance, floor areas scaled by factors drawn uniformly from 0.9 to 1.1) share a 250 kW community PV array, a 500 kWh / 125 kW community battery and one grid connection with a time-of-use tariff (US$0.30 kWh^{-1} on weekdays from 16:00 to 21:00, US$0.08 kWh^{-1} otherwise). The co-simulation contains 18 components and 24 connections, including a fan-in that sums the ten building loads and a delayed connection that feeds the battery's state of charge back to the dispatch controller. Four strategies were compared for the week of 17–23 July: *baseline* (PV only, occupied cooling set-point 24 °C); *precool* (22 °C for three hours before the peak, 26.5 °C during it); *battery* (rule-based dispatch that stores PV surplus, charges at night, discharges during the peak and shaves imports above 120 kW); and *coordinated* (both). Figure 5 and Table 3 show the results.

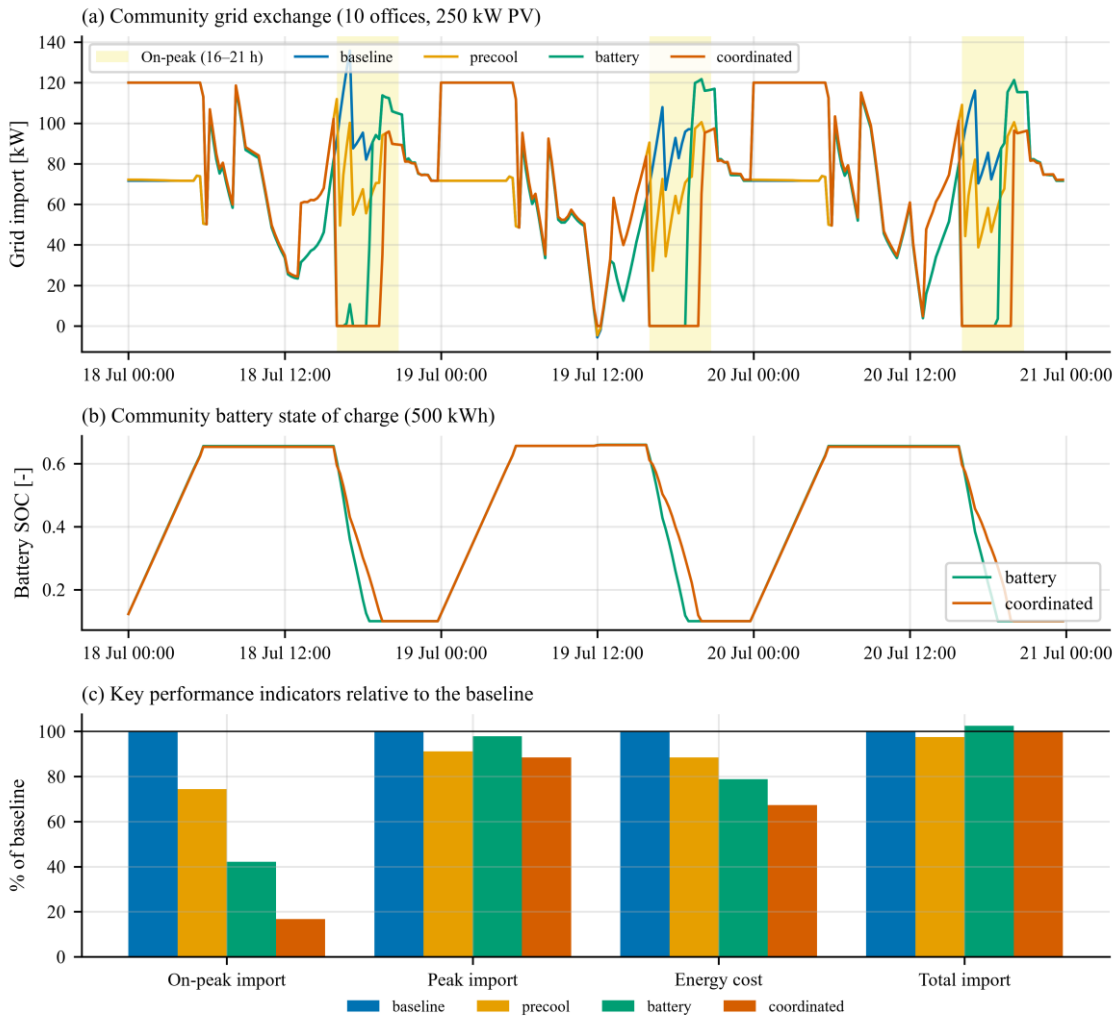


Fig. 5 Case A: (a) community grid import for the four strategies (on-peak periods shaded); (b) battery state of charge; (c) key performance indicators relative to the baseline.

Table 3 Case A results for one summer week (10 buildings)

Strategy	Load (MWh)	Grid import (MWh)	On-peak import (MWh)	Peak import (kW)	Energy cost (US$)	Discomfort (K·h per bldg.)	Run time (s)
Baseline	18.51	10.63	2.50	136	1401	0.0	25
Pre-cooling	18.25	10.36	1.86	124	1238	3.5	25
Battery	18.51	10.90	1.05	133	1104	0.0	26
Coordinated	18.25	10.63	0.42	120	942	3.5	27

Pre-cooling alone reduced on-peak import by 26% and energy cost by 12%; because the pre-cooling hours (13:00–16:00) coincide with high PV output, the additional cooling load was largely covered by PV, and the community peak import decreased by 9%. The battery reduced on-peak import by 58% and energy cost by 21%, while total grid import increased by 3%

because of conversion losses and night-time charging. Coordinating both measures reduced on-peak import by 83% and energy cost by 33% relative to the baseline, and the peak import decreased by 12%: pre-cooling lowers the on-peak energy the battery must supply, so the stored energy lasts through the whole peak window. The price is a modest comfort penalty during the set-up period (3.5 K·h above 26 °C per building and week). Each strategy took about 26 s to simulate. Studying such interactions – here between the thermal mass of ten detailed building models, PV, storage and a tariff – is what a co-simulation framework should make routine; replacing the rule-based dispatch by model-predictive or RL control requires changing a single component.

5.2 Case B: LLM-in-the-loop supervisory control

LLMs are being explored as interfaces between occupants or operators and building controls (Liu et al. 2025; Zhang and Chen 2024; Jiang et al. 2024; Ahn et al. 2023; Zhu and Li 2025). Case B places an LLMSupervisor above the small office: every hour it receives an operator intent and the current core-zone and outdoor temperatures, electricity price and building power, and returns cooling and heating set-points, which EnergyPlus applies at its 15-min steps. Because the purpose of this case is to characterize the integration layer – multi-rate scheduling, prompt construction, parsing, guardrails, logging and fault tolerance – independently of any particular model, the decisions were produced by the framework's offline stand-in back-end, a deterministic keyword-rule policy that answers in the same JSON format as an LLM. The same scenario runs with Claude, OpenAI-compatible or local Ollama models by changing one argument; evaluating the decision quality of specific LLMs is left to future work.

Three intents were tested for five weekdays: a nursing home prioritizing comfort, a normally operated office, and cost minimization during the evening peak (Fig. 6a,b). The supervisor's decisions propagated as intended: the mean occupied-hour cooling set-point was 23.7, 24.2

and 26.2 °C, and the building used 1415, 1394 and 1299 kWh, respectively. Every hourly exchange (prompt, response, applied value, guardrail notes, latency) was recorded in the event log; an example exchange is reproduced in Appendix A.

Robustness was then tested by wrapping the back-end in a fault injector that, with a total probability between 0 and 50%, replaced answers by prose without JSON, out-of-range values (+40 K), answers with extraneous keys, refusals or timeouts (Fig. 6c,d). The co-simulation never failed. At the highest fault rate, 70 of 120 decisions were corrupted; the guardrails held the previous set-point 41 times and clipped 32 out-of-range proposals, so that the applied cooling set-point always stayed within [26.0, 30.0] °C, inside the permitted 21–30 °C band. The effect on the building was limited to a -1.1% change in energy use and a mean absolute zone-temperature deviation of 0.30 K relative to the fault-free run. This shows how co-simulation can be used to stress-test AI-in-the-loop control before deployment.

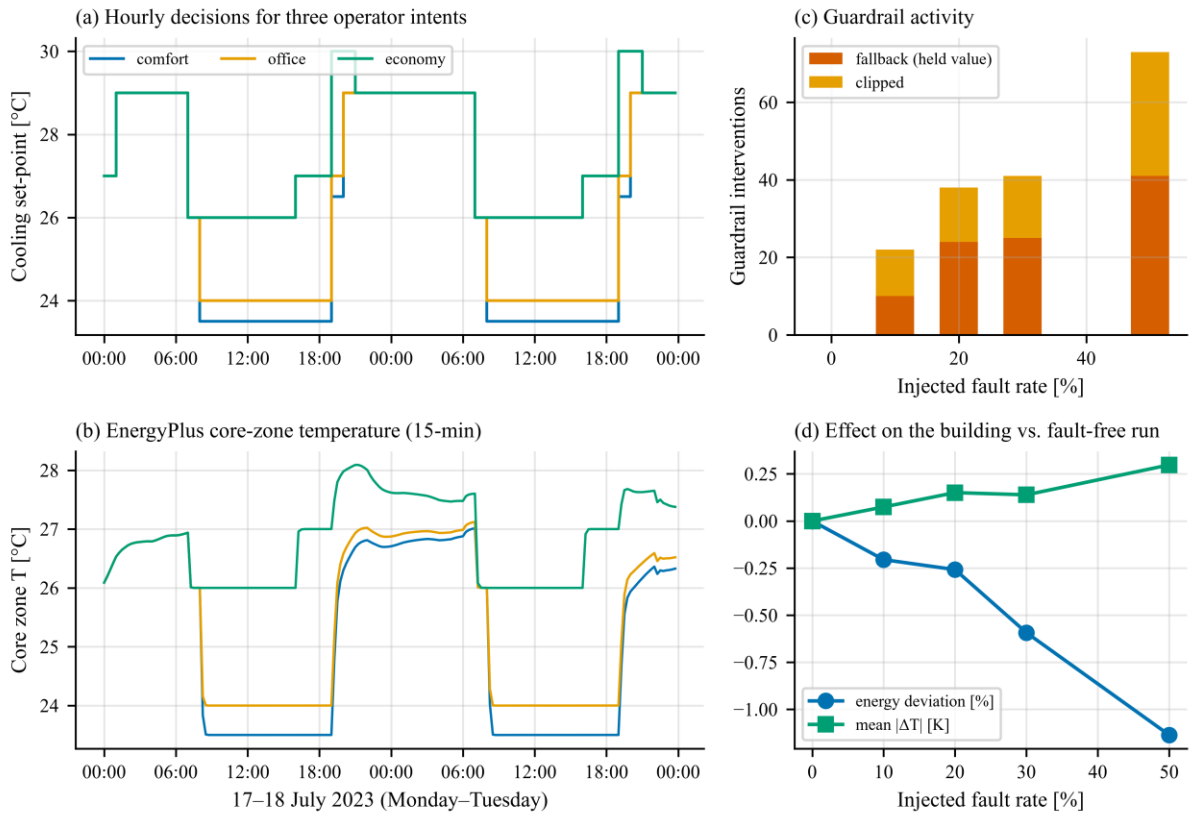


Fig. 6 Case B: (a) hourly cooling set-points chosen by the supervisor for three operator intents and (b) resulting core-zone temperatures; (c) guardrail interventions and (d) effect on the building under increasing rates of injected faults.

5.3 Case C: from travel behavior to apartment cooling loads

Case C revisits the interdisciplinary question of the prototype (Zhang et al. 2023) – how commuting patterns shape residential loads – with a corrected occupancy method and a richer building model, in the spirit of recent mobility-building studies (Liu and Dong 2025). The 23 dwelling units of a mid-rise apartment prototype (sized with the weather file's extreme periods for Tucson) were each assigned a household from the NHTS data. A HouseholdOccupancy component sets the number of people in every unit at every 15-min step through EnergyPlus' per-unit occupancy actuators. Two thermostat policies were compared, both writing the cooling set-point of every unit through EnergyPlus' zone temperature-control actuators: *fixed* (23.9 °C all day) and *smart* (an occupancy-responsive set-up to 26.7 °C whenever a unit is empty). The prototype's own door-switch thermostat reset (an EMS program) and its window

ventilation were removed in a pre-processing step so that the two policies are compared on equal terms – an example of the model-editing hook. Four household groups were simulated – long commute (proxy ≥ 30 min), short commute (≤ 10 min), EV/hybrid owners and other households – each with three independent household samples and both policies, i.e., 24 one-week apartment simulations, four of which ran in parallel in each co-simulation. For each sample the fixed and smart buildings were driven by the same occupancy component, giving a paired comparison.

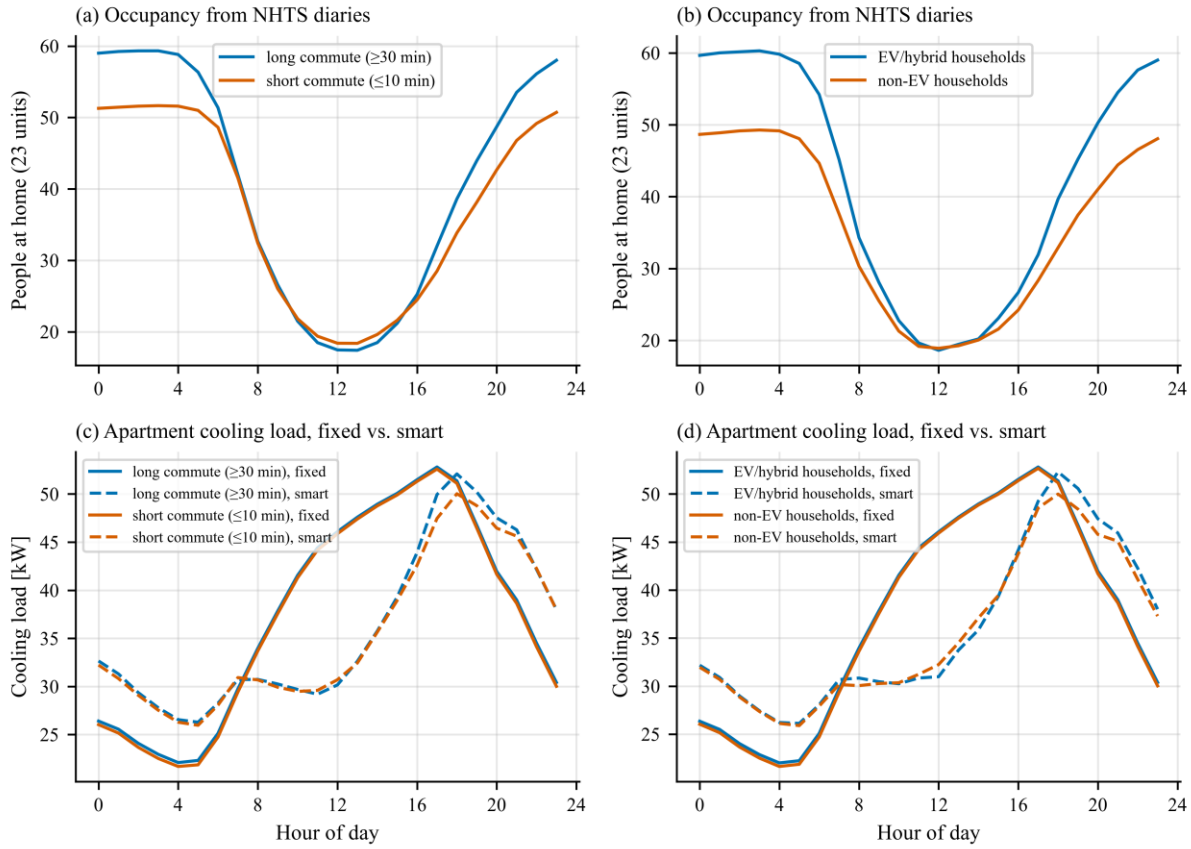


Fig. 7 Case C: average daily profiles of people at home (a, b) and of the apartment cooling load under fixed and occupancy-responsive (smart) thermostats (c, d) for long- versus short-commute households (left) and EV/hybrid-owning versus other households (right); averages over three household samples and seven days.

Table 4 Case C results per household group (simulation results: mean of three household samples ± one standard deviation, one week, 23 units; household statistics: whole NHTS group)

Group	Household size (persons)	Time with nobody home (% of day)	Cooling load, fixed (MWh)	Cooling load, smart (MWh)	Cooling-load saving (%)	Cooling-electricity saving (%)	Peak/valley, fixed	Peak/valley, smart
Long commute	2.37	21.9	6.28	6.03	4.1 ± 0.7	5.7 ± 0.8	2.39	1.99
Short commute	2.22	19.4	6.23	5.95	4.5 ± 0.6	6.0 ± 0.6	2.43	1.93
EV/hybrid	2.40	20.5	6.28	6.04	3.8 ± 0.5	5.3 ± 0.5	2.40	2.00
Non-EV	2.21	20.2	6.23	5.98	4.0 ± 0.7	5.6 ± 0.7	2.43	1.93

Figure 7 and Table 4 summarize the results. The occupancy profiles reflect the travel diaries: in all groups the number of people at home falls from about 06:00 to a minimum around noon and recovers in the evening; long-commute and EV/hybrid-owning households, which are larger on average, have more people at home at night but slightly fewer at midday. Under fixed thermostats, the groups' cooling loads differed little, because envelope and solar gains dominate the cooling demand of the apartments in a Tucson summer and occupants mainly add internal gains. Occupancy-responsive thermostats couple the load directly to presence. They saved 5.7% of the cooling electricity for long-commute and 6.0% for short-commute households (a difference smaller than the variability between household samples), and 5.3% versus 5.6% for EV/hybrid-owning versus other households (a difference smaller than the variability between household samples). The NHTS statistics in Table 4 explain why commuting alone is a weak predictor: a set-up is possible only when a unit is completely empty, which depends on household size as much as on travel. Long-commute households are absent longer per person but are also larger (2.37 versus 2.22 persons), and the share of the day with nobody home was 21.9% versus 19.4%. Smart thermostats also reshaped the daily load profile: they strongly reduced the midday load of empty units, while part of that saving reappeared as a recovery load when occupants return in the evening and at night, shifting the

peak hour from 17:00 to 18:00 and changing the ratio between the highest and the lowest hourly mean load from 2.4 to 2.0 on average – a shift that matters for grid operators when such devices are deployed at scale. The value of a technology, and thus of programmes that subsidize it, can therefore depend on how households travel and how they are composed, which in turn correlate with income and location; such interactions remain invisible without coupling building and mobility models. The results are illustrative: they rest on one-day diaries, a simple presence rule, three household samples per group and one climate, and should be read as a demonstration of the workflow rather than as general conclusions.

6 Discussion

6.1 Usability and lessons learned

Py-CoSim 0.x required about 470 lines of notebook code – building classes with callback functions, threads, file-based messaging and plotting – for two buildings and three Python components. In the new framework the pre-cooling comparison of Example 3 takes 65 lines including key-performance-indicator computation and plotting, and the ten-building community of Case A 122 lines; the framework itself comprises about 4,100 lines of library code, which users do not need to read, and is covered by an automated test suite. Equally important are the lessons on the EnergyPlus API that the framework now encapsulates: simulations must not share a process; time stamps must be derived from zone time-step numbers; outputs should be read after zone reporting to align with EnergyPlus' own reports; meter values refer to the preceding interval; daylight-saving time silently shifts schedules relative to standard-time clocks; and models must be synchronized with the co-simulation window and time step. Each of these produced subtle, silent errors during development, and each is now handled automatically or reported explicitly.

6.2 Limitations

Several limitations remain. Values published by state-based components describe the interval that just ended, whereas tariffs, batteries and controllers act on the interval that starts; aggregations such as the grid balance of Case A therefore combine adjacent intervals at tariff changes, which we estimated to bias a week's energy cost by about 2% at a 15-min step (the convention is documented, and delayed connections can align it). The master uses fixed communication steps without rollback or iteration, so strongly coupled feedthrough loops must be broken with delays and their accuracy depends on the step size (Trčka et al. 2009); the time resolution of EnergyPlus coupling is the zone time step. EnergyPlus support has been verified with version 22.2 on Windows; the implementation relies only on the basic runtime and data-exchange functions of the API, but other versions and operating systems have so far been exercised only by the EnergyPlus-free part of the test suite. All workers run on one machine, so very large communities are limited by memory (each EnergyPlus instance needs tens to a few hundred megabytes). Python components share the master's interpreter, so computationally heavy Python models do not run in parallel unless they release the global interpreter lock. Case B used a deterministic stand-in rather than an LLM, and Case C relies on one-day travel diaries and a simple presence rule.

6.3 Future work

Planned extensions include FMU import so that Modelica models can participate as components (Blochwitz et al. 2011), distributed workers on several machines, bridges to grid co-simulation federations (Hardy et al. 2024) and to benchmarking services (Blum et al. 2021), hardware-in-the-loop execution against building automation systems, benchmarks of LLM supervisors with real models using the record-and-replay facility, and data-predictive and model-predictive control templates (Zhang 2021; Drgoňa et al. 2020).

7 Conclusions

Py-CoSim makes co-simulation of EnergyPlus buildings with arbitrary Python models practical for building researchers. Its component abstraction treats EnergyPlus models, physical and data-driven models, controllers, LLM agents and RL agents uniformly; a lock-step protocol turns EnergyPlus into a steppable component running in isolated, parallel worker processes; and a graph-based master algorithm with an in-memory data bus gives co-simulations explicit, verifiable and reproducible semantics. Verification showed bit-identical agreement with stand-alone EnergyPlus and exact equivalence of Python actuation, and parallel workers accelerated multi-building runs by up to 2.9 times with identical results. Three case studies – coordinated pre-cooling and storage in a ten-building community, LLM supervisory control stress-tested with injected faults, and travel-diary-driven occupancy in 23 apartments – illustrate the range of interdisciplinary questions the framework supports. Py-CoSim is open source, documented and accompanied by nine runnable examples intended as starting points for new studies.

Code availability. Py-CoSim, the examples and the scripts that reproduce every result and figure of this paper are open source under the MIT license (repository link withheld for double-blind review; it is given on the title page).

Appendix A Example of a recorded LLM exchange (Case B)

The following prompt was generated by the LLMSupervisor at one hourly decision of the nursing-home run; the system prompt instructs the model to answer with a single JSON object containing clg_sp and htg_sp within their permitted ranges and a short reason. The response and the applied set-points are stored with the prompt in the event log.

```
PROMPT: OPERATOR INTENT: This building is a nursing home. Residents are elderly; keep them
comfortable at all times.

STATE: {"time": "2023-07-17 10:00", "weekday": "Monday", "observations": {"T_zone": 23.5,
"T_out": 34.4, "price": 0.08, "P_elec": 18.52}, "observation_meaning": {"T_zone": "core
zone air temperature [C]", "T_out": "outdoor air temperature [C]", "price": "electricity
```

price [$/kWh]", "P_elec": "building electric power [kW]"}, "current_setpoints": {"clg_sp": 23.5, "htg_sp": 22.5}, "occupied": true}

RESPONSE: {"clg_sp": 23.5, "htg_sp": 22.5, "reason": "rule profile 'comfort'"}

APPLIED: {'clg_sp': 23.5, 'htg_sp': 22.5}

References

Ahn KU, Kim DW, Cho HM, et al. (2023). Alternative approaches to HVAC control of Chat Generative Pre-Trained Transformer (ChatGPT) for autonomous building system operations. Buildings, 13: 2680.

Allegrini J, Orehounig K, Mavromatidis G, et al. (2015). A review of modelling approaches and tools for the simulation of district-scale energy systems. Renewable and Sustainable Energy Reviews, 52: 1391–1404.

Benne K, Shapinsky T, Petersen A (2024). Alfalfa virtual building service: Software engineering best practices applied to runtime interaction with building energy models. Presentation at IBPSA-USA SimBuild 2024 Conference, Denver, CO, USA. NREL/PR-5500-89738, National Renewable Energy Laboratory.

Bernal W, Behl M, Nghiem TX, et al. (2012). MLE+: A tool for integrated design and deployment of energy efficient building controls. In: Proceedings of the 4th ACM Workshop on Embedded Sensing Systems for Energy-Efficiency in Buildings (BuildSys '12), Toronto, Canada, pp. 123–130.

Blochwitz T, Otter M, Arnold M, et al. (2011). The Functional Mockup Interface for tool independent exchange of simulation models. In: Proceedings of the 8th International Modelica Conference, Dresden, Germany, pp. 105–114.

Blum D, Arroyo J, Huang S, et al. (2021). Building optimization testing framework (BOPTEST) for simulation-based benchmarking of control strategies in buildings. Journal of Building Performance Simulation, 14: 586–610.

Bonnema E, Allen A, Lee E, et al. (2025). Flexible integration of diverse HVAC technologies in EnergyPlus via Python-enabled workflows. Technical Report, NREL/TP-5500-85628, National Renewable Energy Laboratory, USA.

Campoy-Nieves A, Manjavacas A, Jiménez-Raboso J, et al. (2025). Sinergym – A virtual testbed for building energy optimization with reinforcement learning. Energy and Buildings, 327: 115075.

Chen Y, Hong T, Luo X (2018). An agent-based stochastic occupancy simulator. Building Simulation, 11: 37–49.

Chinde V, Woldekidan K, Bonnema E, et al. (2025). Integration of a grey-box refrigerated case model in EnergyPlus via Python plugin. Building Simulation, 18: 3119–3132.

Crawley DB, Lawrie LK, Winkelmann FC, et al. (2001). EnergyPlus: Creating a new-generation building energy simulation program. Energy and Buildings, 33: 319–331.

Dai X, Chen R, Guan S, et al. (2025). BuildingGym: An open-source toolbox for AI-based building energy management using reinforcement learning. Building Simulation, 18: 1909–1927.

Deru M, Field K, Studer D, et al. (2011). U.S. Department of Energy commercial reference building models of the national building stock. Technical Report, NREL/TP-5500-46861, National Renewable Energy Laboratory, USA.

Dobos AP (2014). PVWatts version 5 manual. Technical Report, NREL/TP-6A20-62641, National Renewable Energy Laboratory, USA.

DOE (2022). EnergyPlus Version 22.2 Input Output Reference: Group – Python Plugins / Python EMS. U.S. Department of Energy. Available at https://bigladdersoftware.com/epx/docs/22-2/input-output-reference/group-python-plugins.html. Accessed 27 Oct 2026.

Drgoňa J, Arroyo J, Cupeiro Figueroa I, et al. (2020). All you need to know about model predictive control for buildings. Annual Reviews in Control, 50: 190–232.

El Kontar R, Polly B, Charan T, et al. (2020). URBANopt: An open-source software development kit for community and urban district energy modeling. In: Proceedings of SimBuild 2020, ASHRAE/IBPSA-USA Building Simulation Conference.

FHWA (2017). 2017 National Household Travel Survey. U.S. Department of Transportation, Federal Highway Administration. Available at https://nhts.ornl.gov. Accessed 27 Oct 2026.

Gomes C, Thule C, Broman D, et al. (2018). Co-simulation: A survey. ACM Computing Surveys, 51: 49.

Hardy TD, Palmintier B, Top PL, et al. (2024). HELICS: A co-simulation framework for scalable multi-domain modeling and analysis. IEEE Access, 12: 24325–24347.

Hong T, Chen Y, Luo X, et al. (2020). Ten questions on urban building energy modeling. Building and Environment, 168: 106508.

Hong T, Langevin J, Sun K (2018). Building simulation: Ten challenges. Building Simulation, 11: 871–898.

Hong T, Lee SH, Zhang W, et al. (2025). Informing electrification strategies of residential neighborhoods with urban building energy modeling. Building Simulation, 18: 3–18.

Jiang G, Ma Z, Zhang L, et al. (2024). EPlus-LLM: A large language model-based computing platform for automated building energy modeling. Applied Energy, 367: 123431.

Jiménez-Raboso J, Campoy-Nieves A, Manjavacas-Lucas A, et al. (2021). Sinergym: A building simulation and control framework for training reinforcement learning agents. In: Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '21), Coimbra, Portugal, pp. 319–323.

Kahn AB (1962). Topological sorting of large networks. Communications of the ACM, 5: 558–562.

Kübler R, Schiehlen W (2000). Two methods of simulator coupling. Mathematical and Computer Modelling of Dynamical Systems, 6: 93–113.

Li H, Hong T (2025). A digital twin platform for building performance monitoring and optimization: Performance simulation and case studies. Building Simulation, 18: 1561–1579.

Liu M, Zhang L, Chen J, et al. (2025). Large language models for building energy applications: Opportunities and challenges. Building Simulation, 18: 225–234.

Liu Y, Dong B (2025). Urban scale vehicle-to-building-to-grid integration leveraging human mobility modeling for enhanced grid flexibility. Building Simulation, 18: 3069–3095.

Moriyama T, De Magistris G, Tatsubori M, et al. (2018). Reinforcement learning testbed for power-consumption optimization. In: Li L, Hasegawa K, Tanaka S (eds), Methods and Applications for Modeling and Simulation of Complex Systems (AsiaSim 2018). Communications in Computer and Information Science, Vol. 946. Singapore: Springer. pp. 45–59.

Neukomm M, Nubbe V, Fares R (2019). Grid-interactive efficient buildings technical report series: Overview of research challenges and gaps. Technical Report, NREL/TP-5500-75470, National Renewable Energy Laboratory, USA.

Nouidui TS, Wetter M, Zuo W (2014). Functional mock-up unit for co-simulation import in EnergyPlus. Journal of Building Performance Simulation, 7: 192–202.

Nweye K, Kaspar K, Buscemi G, et al. (2025). CityLearn v2: Energy-flexible, resilient, occupant-centric, and carbon-aware management of grid-interactive communities. Journal of Building Performance Simulation, 18: 17–38.

Page J, Robinson D, Morel N, et al. (2008). A generalised stochastic model for the simulation of occupant presence. Energy and Buildings, 40: 83–98.

Palmintier B, Krishnamurthy D, Top P, et al. (2017). Design of the HELICS high-performance transmission-distribution-communication-market co-simulation framework. In: Proceedings of 2017 Workshop on Modeling and Simulation of Cyber-Physical Energy Systems (MSCPES), pp. 1–6.

Reinhart CF, Cerezo Davila C (2016). Urban building energy modeling – A review of a nascent field. Building and Environment, 97: 196–202.

Scharnhorst P, Schubnel B, Fernández Bandera C, et al. (2021). Energym: A building model library for controller benchmarking. Applied Sciences, 11: 3518.

Steinbrink C, Blank-Babazadeh M, El-Ama A, et al. (2019). CPES testing with mosaik: Co-simulation planning, execution and analysis. Applied Sciences, 9: 923.

Towers M, Kwiatkowski A, Balis J, et al. (2025). Gymnasium: A standard interface for reinforcement learning environments. In: Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track.

Trčka M, Hensen JLM, Wetter M (2009). Co-simulation of innovative integrated HVAC systems in buildings. Journal of Building Performance Simulation, 2: 209–230.

Vázquez-Canteli JR, Kämpf J, Henze G, et al. (2019). CityLearn v1.0: An OpenAI gym environment for demand response with deep reinforcement learning. In: Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '19), New York, NY, USA, pp. 356–357.

Wang Z, Hong T (2020). Reinforcement learning for building controls: The opportunities and challenges. Applied Energy, 269: 115036.

Wetter M (2011). Co-simulation of building energy and control systems with the Building Controls Virtual Test Bed. Journal of Building Performance Simulation, 4: 185–203.

Wetter M, Benne K, Gautier A, et al. (2020). Lifting the garage door on Spawn, an open-source BEM-controls engine. In: Proceedings of SimBuild 2020, ASHRAE/IBPSA-USA Building Simulation Conference.

Wetter M, Zuo W, Nouidui TS, et al. (2014). Modelica Buildings library. Journal of Building Performance Simulation, 7: 253–270.

Yan D, O'Brien W, Hong T, et al. (2015). Occupant behavior modeling for building performance simulation: Current state and future challenges. Energy and Buildings, 107: 264–278.

Zhang L (2021). Data-driven building energy modeling with feature selection and active learning for data predictive control. Energy and Buildings, 252: 111436.

Zhang L, Chen Z (2024). Large language model-based interpretable machine learning control in building energy systems. Energy and Buildings, 313: 114278.

Zhang L, Haroon SM, Ryan A (2023). Py-CoSim: Python-based building energy co-simulation infrastructure. SSRN preprint, https://doi.org/10.2139/ssrn.4572925.

Zhang L, Wen J, Li Y, et al. (2021). A review of machine learning in building load prediction. Applied Energy, 285: 116452.

Zhang T, Ardakanian O (2020). COBS: COmprehensive Building Simulator. In: Proceedings of the 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '20), pp. 314–315.

Zhang Z, Lam KP (2018). Practical implementation and evaluation of deep reinforcement learning control for a radiant heating system. In: Proceedings of the 5th Conference on Systems for Built Environments (BuildSys '18), Shenzhen, China, pp. 148–157.

Zhu X, Li H (2025). Heating, ventilation, and air conditioning (HVAC) temperature and humidity control optimization based on large language models (LLMs). Energies, 18: 1813.